TI – 2 SPRINT 3

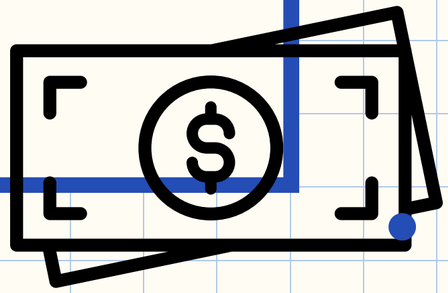
FINANCIAL CONTROLLER

PROJECT PRESENTATION

HEITOR LM, MATHEUS PROCOPIO, ARTHUR LIMA E NICOLAS DIAS

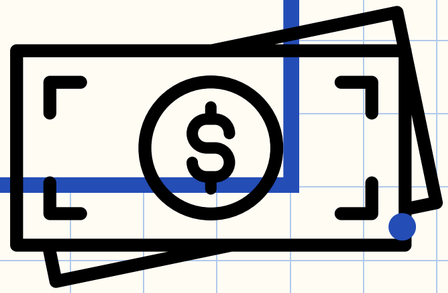
CONTEXTUALIZAÇÃO

- Dificuldade do usuário de organizar seus gastos e acompanhar a saúde financeira
- Muitos aplicativos de controle financeiro exigem entrada manual e categorização manual das despesas.
- Falta de uma solução intuitiva e conversacional, capaz de compreender voz e linguagem natural.



OBJETIVO DO PROJETO

- Desenvolver um assistente financeiro inteligente com interface de chat e reconhecimento de voz, capaz de registrar, classificar e analisar transações automaticamente
- Implementar Speech-to-Text para permitir entrada de dados por voz.
- Utilizar embeddings semânticos para categorização automática das transações
- Gerar relatórios e insights personalizados sobre hábitos financeiros
- Oferecer uma experiência fluida e acessível, promovendo educação e controle financeiro



FLUXO DA APLICAÇÃO

```java
package com.ti2cc.api.routes;

import com.ti2cc.api.controller.CategoryController;

import static spark.Spark.*;

public class CategoryRoutes {

    private final CategoryController controller;

    public CategoryRoutes(CategoryController controller) {
        this.controller = controller;
    }

    public void register() {
        path("/categories", () -> {
            post("", controller::create);
            get("", controller::listByUser);
            get("/:category_id", controller::getById);
            put("/:category_id", controller::update);
            delete("/:category_id", controller::delete);
        });
    }
}
```

```java
package com.ti2cc.api.service;

import com.ti2cc.api.dao.CategoryDAO;
import com.ti2cc.api.model.Category;

import java.util.List;
import java.util.UUID;

public class CategoryService {
    private final CategoryDAO categoryDAO;

    public CategoryService(CategoryDAO categoryDAO) {
        this.categoryDAO = categoryDAO;
    }

    public boolean createCategory(Category category) {
        if (category.getName() == null || category.getName().trim().isEmpty()) {
            System.err.println("Category name cannot be empty.");
            return false;
        }

        if (category.getId() == null) {
            category.setId(UUID.randomUUID());
        }

        return categoryDAO.create(category);
    }

```

DAO

```java
public class CategoryDAO {
    private Connection connection;

    public CategoryDAO(Connection connection) {
        this.connection = connection;
    }

    public boolean create(Category category) {
        StringBuilder sb = new StringBuilder();

        sb.append("INSERT INTO categories (");
        sb.append("id, name, user_id"); // user_id é obrigatório

        if (category.getImagePath() != null) {
            sb.append(", image_path");
        }

        if (category.getMainCategoryId() != null) {
            sb.append(", main_category_id");
        }

        sb.append(") VALUES (");
        sb.append("?, ?, ?"); // parâmetros fixos: id, name e user_id

        if (category.getImagePath() != null) {
            sb.append(", ?");
        }

        if (category.getMainCategoryId() != null) {
            sb.append(", ?");
        }

        sb.append(")");

        String query = sb.toString();

        try (PreparedStatement pstmt = connection.prepareStatement(query)) {
            int paramIndex = 1;

            if (category.getId() == null) {
                category.setId(UUID.randomUUID());
            }

            // parâmetros fixos
            pstmt.setObject(paramIndex++, category.getId());
            pstmt.setString(paramIndex++, category.getName());
            pstmt.setObject(paramIndex++, category.getUserId());

```

MODEL

```java
public class Category {
    private UUID id;
    private UUID userId;
    private String name;
    private String imagePath;
    private UUID mainCategoryId;

    public Category() {
        this(null, null, null, null, null);
    }

    public Category(UUID id, UUID userId, String name, String imagePath, UUID mainCategoryId) {
        this.id = id;
        this.userId = userId;
        this.name = name;
        this.imagePath = imagePath;
        this.mainCategoryId = mainCategoryId;
    }

    public UUID getId() {
        return id;
    }

    public UUID getUserId() {
        return userId;
    }

    public String getName() {
        return name;
    }

    public String getImagePath() {
        return imagePath;
    }

    public UUID getMainCategoryId() {
        return mainCategoryId;
    }

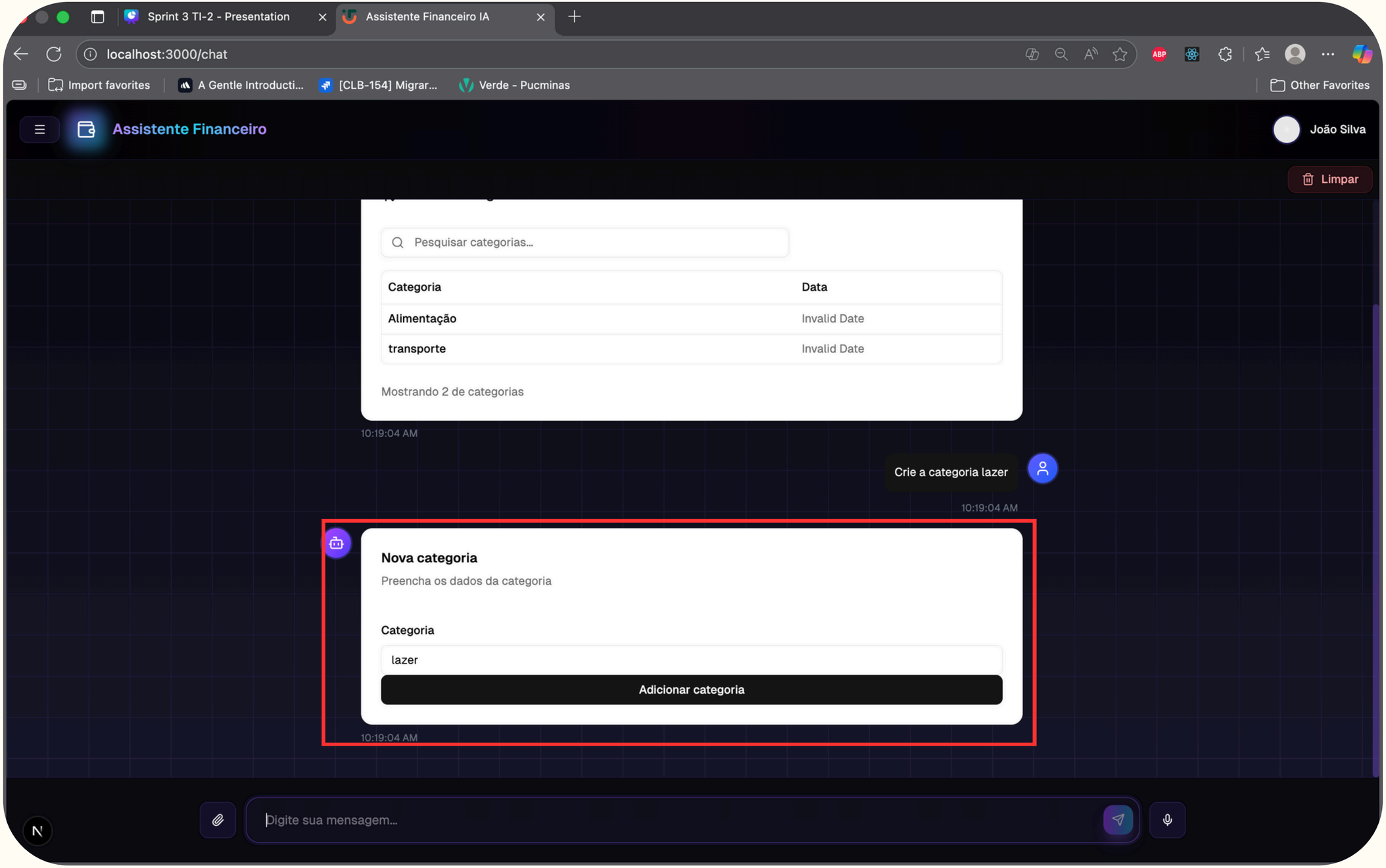
    public void setId(UUID id) {
        this.id = id;
    }

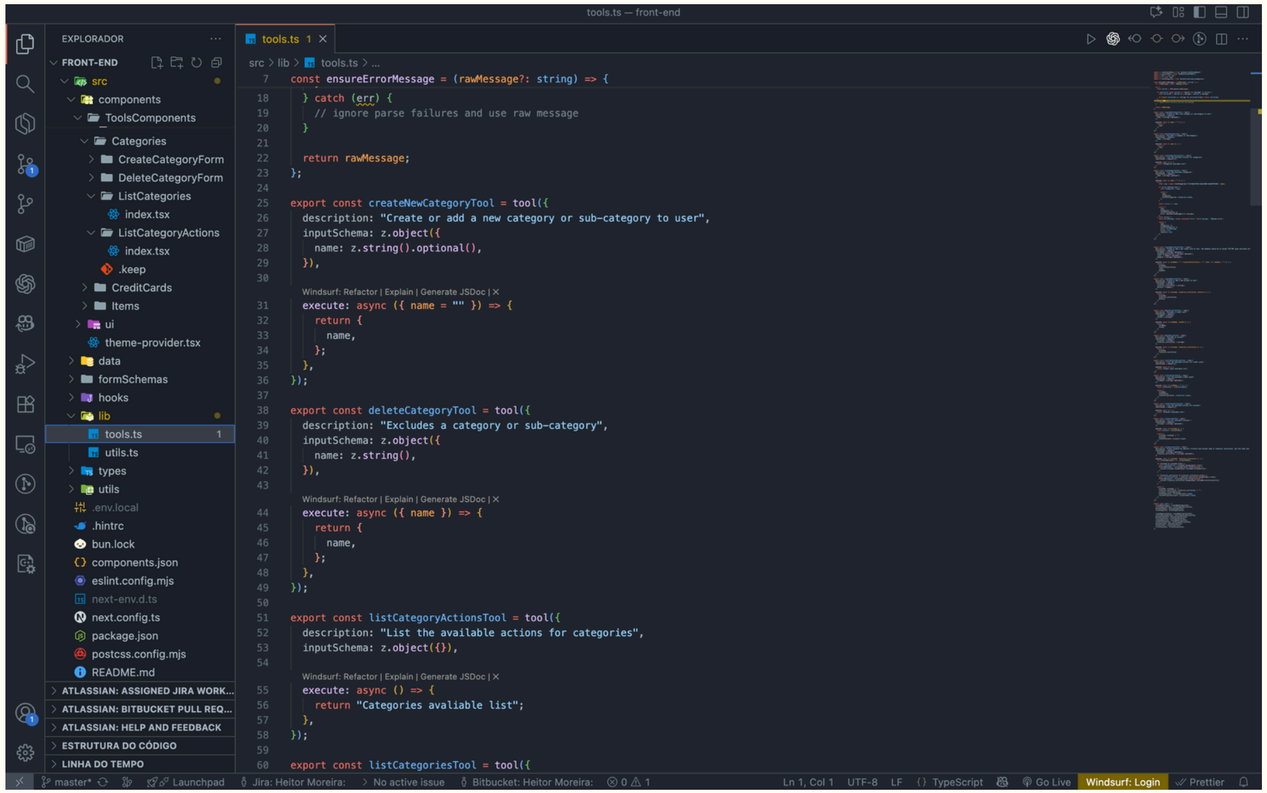
    public void setName(String name) {
        this.name = name;
    }

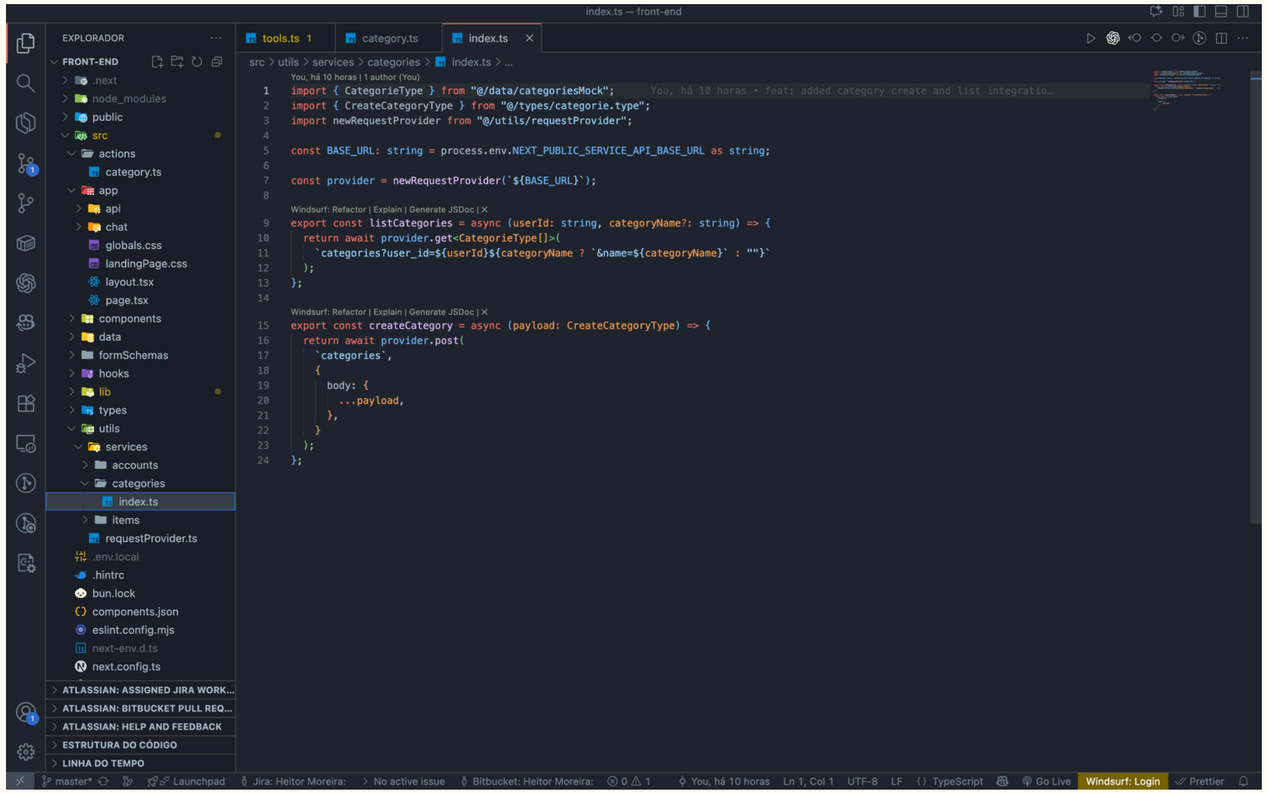
    public void setImagePath(String imagePath) {
        this.imagePath = imagePath;
    }

    public void setMainCategoryId(UUID mainCategoryId) {
        this.mainCategoryId = mainCategoryId;
    }

    public void setUserId(UUID userId) {
        this.userId = userId;
    }
```


Sprint 3 TI-2 - Presentation
Assistente Financeiro IA
localhost:3000/chat
Import favorites
A Gentle Introducti...
[CLB-154] Migrar...
Verde - Pucminas
Other Favorites
Assistente Financeiro
João Silva
Limpar
Pesquisar categorias...
Categoria
Data
Alimentação
Invalid Date
transporte
Invalid Date
Mostrando 2 de categorias
10:19:04 AM
Crie a categoria lazer
10:19:04 AM
Nova categoria
Preencha os dados da categoria
Categoria
lazer
Adicionar categoria
10:19:04 AM
Digite sua mensagem...


tools.ts — front-end
EXPLORADOR
FRONT-END
src
components
ToolsComponents
Categories
CreateCategoryForm
DeleteCategoryForm
ListCategories
index.tsx
ListCategoryActions
index.tsx
.keep
CreditCards
Items
ui
theme-provider.tsx
data
formSchemas
hooks
lib
tools.ts 1
utils.ts
types
utils
.env.local
.hintrc
bun.lock
components.json
eslint.config.mjs
next-env.d.ts
next.config.ts
package.json
postcss.config.mjs
README.md
ATLASSIAN: ASSIGNED JIRA WORK...
ATLASSIAN: BITBUCKET PULL REQ...
ATLASSIAN: HELP AND FEEDBACK
ESTRUTURA DO CÓDIGO
LINHA DO TEMPO
tools.ts 1
src > lib > tools.ts > ...
const ensureErrorMessage = (rawMessage?: string) => {
} catch (err) {
// ignore parse failures and use raw message
}
return rawMessage;
};
export const createNewCategoryTool = tool({
description: "Create or add a new category or sub-category to user",
inputSchema: z.object({
name: z.string().optional(),
}),
Windsurf: Refactor | Explain | Generate JSDoc | ×
execute: async ({ name = "" }) => {
return {
name,
};
},
});
export const deleteCategoryTool = tool({
description: "Excludes a category or sub-category",
inputSchema: z.object({
name: z.string(),
}),
Windsurf: Refactor | Explain | Generate JSDoc | ×
execute: async ({ name }) => {
return {
name,
};
},
});
export const listCategoryActionsTool = tool({
description: "List the available actions for categories",
inputSchema: z.object({}),
Windsurf: Refactor | Explain | Generate JSDoc | ×
execute: async () => {
return "Categories avaliable list";
},
});
export const listCategoriesTool = tool({
master*
Launchpad
Jira: Heitor Moreira:
No active issue
Bitbucket: Heitor Moreira:
0 1
Ln 1, Col 1
UTF-8
LF
TypeScript
Go Live
Windsurf: Login
Prettier


index.ts — front-end
EXPLORADOR
FRONT-END
.next
node_modules
public
src
actions
category.ts
app
api
chat
globals.css
landingPage.css
layout.tsx
page.tsx
components
data
formSchemas
hooks
lib
types
utils
services
accounts
categories
index.ts
items
requestProvider.ts
.env.local
.hintrc
bun.lock
components.json
eslint.config.mjs
next-env.d.ts
next.config.ts
ATLASSIAN: ASSIGNED JIRA WORK...
ATLASSIAN: BITBUCKET PULL REQ...
ATLASSIAN: HELP AND FEEDBACK
ESTRUTURA DO CÓDIGO
LINHA DO TEMPO
tools.ts 1
category.ts
index.ts
src > utils > services > categories > index.ts > ...
You, há 10 horas | 1 author (You)
import { CategorieType } from "@/data/categoriesMock";
You, há 10 horas • feat: added category create and list integratio...
import { CreateCategoryType } from "@/types/categorie.type";
import newRequestProvider from "@/utils/requestProvider";
const BASE_URL: string = process.env.NEXT_PUBLIC_SERVICE_API_BASE_URL as string;
const provider = newRequestProvider(`${BASE_URL}`);
Windsurf: Refactor | Explain | Generate JSDoc | ×
export const listCategories = async (userId: string, categoryName?: string) => {
return await provider.get<CategorieType[]>(
`categories?user_id=${userId}${categoryName ? `&name=${categoryName}` : ""}`
);
};
Windsurf: Refactor | Explain | Generate JSDoc | ×
export const createCategory = async (payload: CreateCategoryType) => {
return await provider.post(
`categories`,
{
body: {
...payload,
},
}
);
};
master*
Launchpad
Jira: Heitor Moreira:
No active issue
Bitbucket: Heitor Moreira:
0 1
You, há 10 horas
Ln 1, Col 1
UTF-8
LF
TypeScript
Go Live
Windsurf: Login
Prettier

FLUXO DA APLICAÇÃO

```java
    public boolean deleteCategory(UUID id) {
        if (id == null) {
            System.err.println("Invalid category ID for deletion.");
            return false;
        }

        return categoryDAO.delete(id);
    }

    public Category findCategoryById(UUID id) {
        if (id == null) {
            System.err.println("Invalid category ID.");
            return null;
        }

        return categoryDAO.getById(id);
    }

    public List<Category> listCategoriesByUserId(UUID userId) {
        if (userId == null) {
            System.err.println("Invalid user ID.");
            return List.of();
        }

        return categoryDAO.getByUserId(userId);
    }
}

```

```java

    public Object listByUser(Request req, Response res) {
        res.type("application/json");
        String userIdParam = req.queryParams("user_id");
        if (userIdParam == null || userIdParam.isBlank()) {
            res.status(400);
            return gson.toJson("Query parameter 'user_id' is required.");
        }

        try {
            UUID userId = UUID.fromString(userIdParam);
            String nameParam = req.queryParams("name");
            String name = (nameParam == null || nameParam.isBlank()) ? null : nameParam;
            String mainCategoryIdParam = req.queryParams("main_category_id");

            UUID mainCategoryId = null;
            if (mainCategoryIdParam != null && !mainCategoryIdParam.isBlank()) {
                mainCategoryId = UUID.fromString(mainCategoryIdParam);
            }

            List<Category> categories = categoryService.listCategories(userId, name, mainCategoryId);
            res.status(200);
            return gson.toJson(categories);
        } catch (IllegalArgumentException e) {
            res.status(400);
            return gson.toJson("Invalid UUID format: " + e.getMessage());
        }
    }

```

```java
    public List<Category> getByFilters(UUID userId, String name, UUID mainCategoryId) {
        List<Category> list = new ArrayList<>();
        StringBuilder query = new StringBuilder("SELECT * FROM categories WHERE user_id = ?");

        boolean hasNameFilter = name != null && !name.trim().isEmpty();

        if (hasNameFilter) {
            query.append(" AND name ILIKE ?");
        }

        if (mainCategoryId != null) {
            query.append(" AND main_category_id = ?");
        }

        try (PreparedStatement pstmt = connection.prepareStatement(query.toString())) {
            int paramIndex = 1;
            pstmt.setObject(paramIndex++, userId);

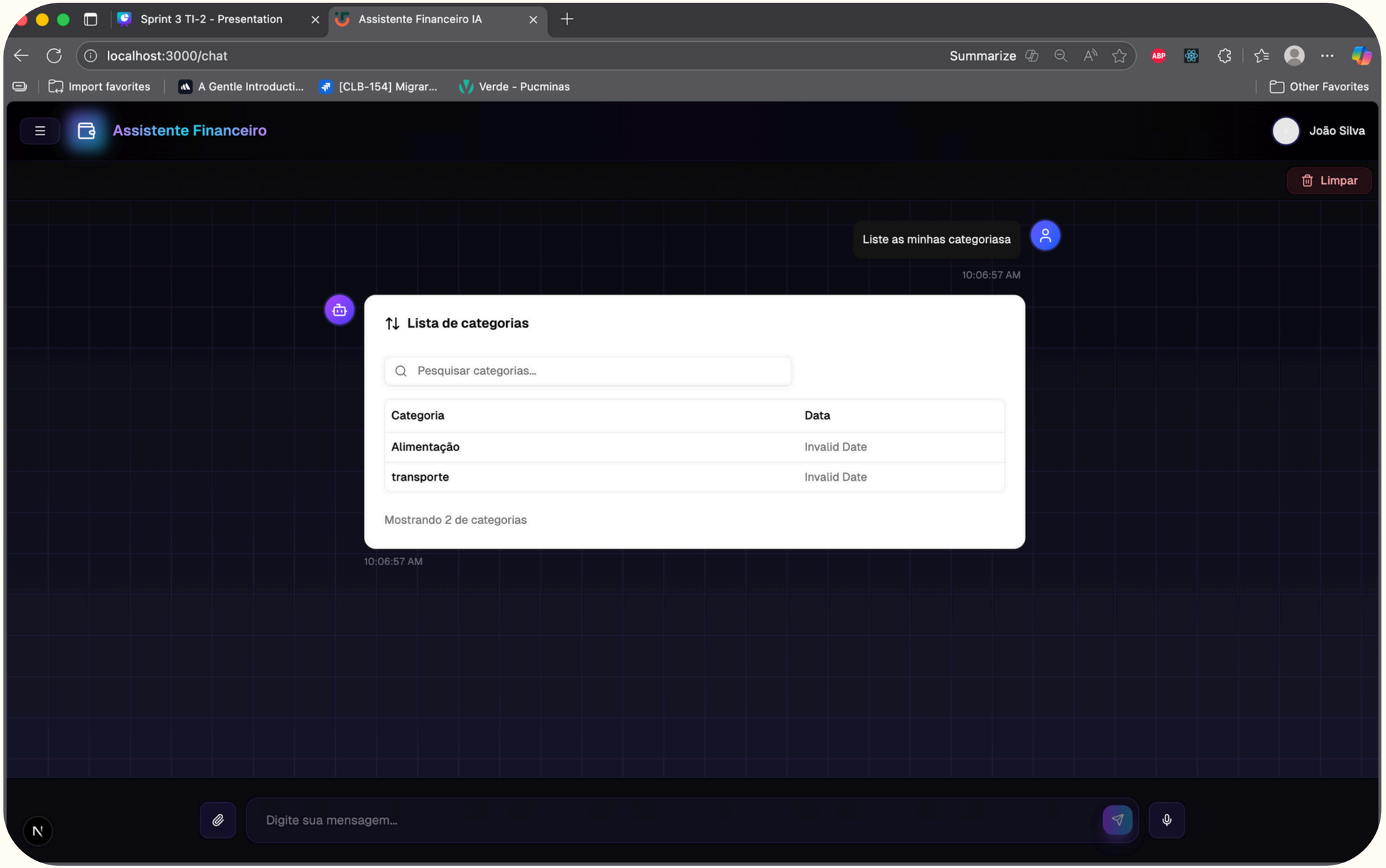
            if (hasNameFilter) {
                pstmt.setString(paramIndex++, "%" + name.trim() + "%");
            }

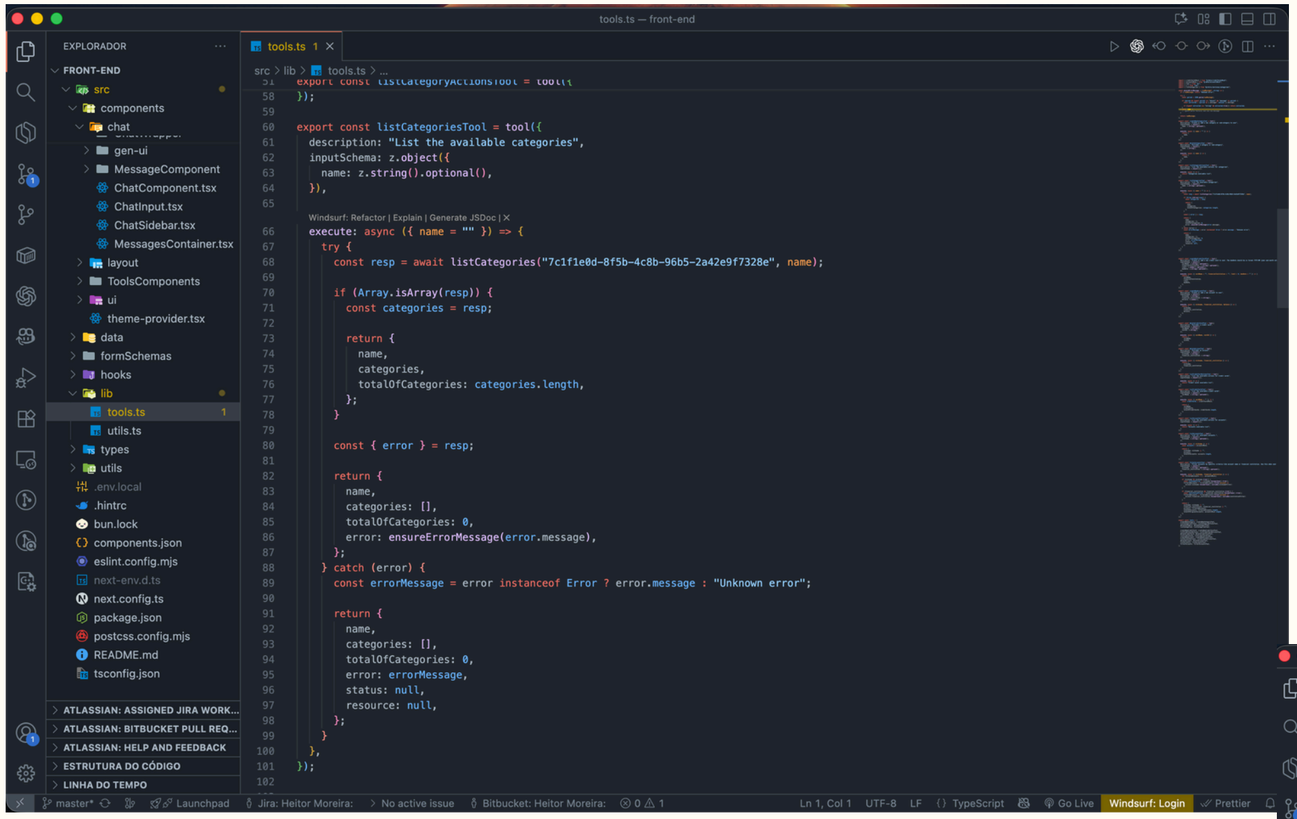
            if (mainCategoryId != null) {
                pstmt.setObject(paramIndex, mainCategoryId);
            }

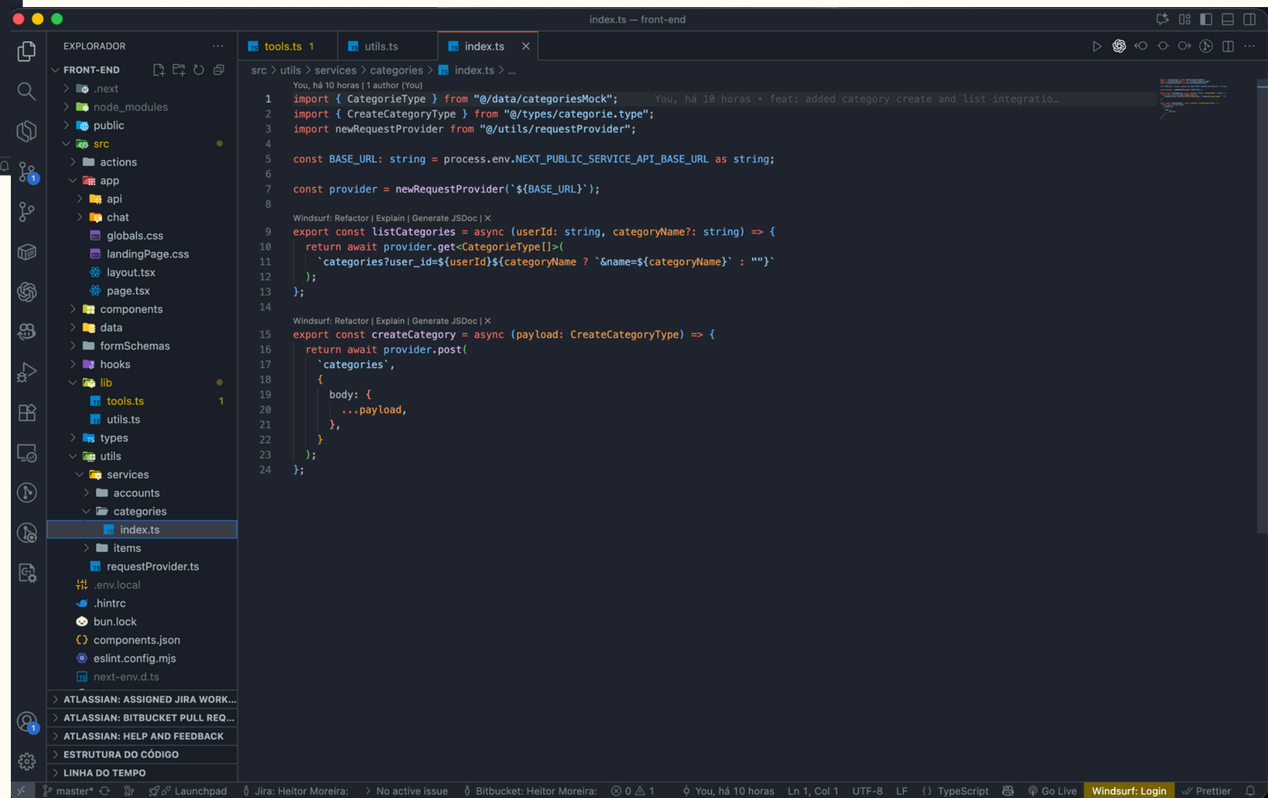
            ResultSet rs = pstmt.executeQuery();
            while (rs.next()) {
                Category c = new Category(
                        (UUID) rs.getObject("id"),
                        (UUID) rs.getObject("user_id"),
                        rs.getString("name"),
                        rs.getString("image_path"),
                        (UUID) rs.getObject("main_category_id"));
                list.add(c);
            }
        } catch (SQLException e) {
            System.err.println("Fail to list categories by filters: " + e.getMessage());
        }

        return list;
    }
}

```


Sprint 3 TI-2 - Presentation
Assistente Financeiro IA
localhost:3000/chat
Summarize
Import favorites
A Gentle Introducti...
[CLB-154] Migrar...
Verde - Pucminas
Other Favorites
Assistente Financeiro
João Silva
Limpar
Liste as minhas categoriasa
10:06:57 AM
Lista de categorias
Pesquisar categorias...
Categoria
Data
Alimentação
Invalid Date
transporte
Invalid Date
Mostrando 2 de categorias
10:06:57 AM
Digite sua mensagem...


tools.ts — front-end
EXPLORADOR
FRONT-END
src
components
chat
gen-ui
MessageComponent
ChatComponent.tsx
ChatInput.tsx
ChatSidebar.tsx
MessagesContainer.tsx
layout
ToolsComponents
ui
theme-provider.tsx
data
formSchemas
hooks
lib
tools.ts
utils.ts
types
utils
.env.local
.hintrc
bun.lock
components.json
eslint.config.mjs
next-env.d.ts
next.config.ts
package.json
postcss.config.mjs
README.md
tsconfig.json
ATLASSIAN: ASSIGNED JIRA WORK...
ATLASSIAN: BITBUCKET PULL REQ...
ATLASSIAN: HELP AND FEEDBACK
ESTRUTURA DO CÓDIGO
LINHA DO TEMPO
tools.ts 1
src > lib > tools.ts > ...
export const listCategoryActionsTool = tool({
});
export const listCategoriesTool = tool({
description: "List the available categories",
inputSchema: z.object({
name: z.string().optional(),
}),
Windsurf: Refactor | Explain | Generate JSDoc |
execute: async ({ name = "" }) => {
try {
const resp = await listCategories("7c1f1e0d-8f5b-4c8b-96b5-2a42e9f7328e", name);
if (Array.isArray(resp)) {
const categories = resp;
return {
name,
categories,
totalOfCategories: categories.length,
};
}
const { error } = resp;
return {
name,
categories: [],
totalOfCategories: 0,
error: ensureErrorMessage(error.message),
};
} catch (error) {
const errorMessage = error instanceof Error ? error.message : "Unknown error";
return {
name,
categories: [],
totalOfCategories: 0,
error: errorMessage,
status: null,
resource: null,
};
}
},
});
master*
Launchpad
Jira: Heitor Moreira:
No active issue
Bitbucket: Heitor Moreira:
0 1
Ln 1, Col 1
UTF-8
LF
TypeScript
Go Live
Windsurf: Login
Prettier


index.ts — front-end
EXPLORADOR
FRONT-END
.next
node_modules
public
src
actions
app
api
chat
globals.css
landingPage.css
layout.tsx
page.tsx
components
data
formSchemas
hooks
lib
tools.ts
utils.ts
types
utils
services
accounts
categories
index.ts
items
requestProvider.ts
.env.local
.hintrc
bun.lock
components.json
eslint.config.mjs
next-env.d.ts
ATLASSIAN: ASSIGNED JIRA WORK...
ATLASSIAN: BITBUCKET PULL REQ...
ATLASSIAN: HELP AND FEEDBACK
ESTRUTURA DO CÓDIGO
LINHA DO TEMPO
tools.ts 1
utils.ts
index.ts
src > utils > services > categories > index.ts > ...
You, há 10 horas | 1 author (You)
import { CategorieType } from "@/data/categoriesMock";
You, há 10 horas • feat: added category create and list integratio...
import { CreateCategoryType } from "@/types/categorie.type";
import newRequestProvider from "@/utils/requestProvider";
const BASE_URL: string = process.env.NEXT_PUBLIC_SERVICE_API_BASE_URL as string;
const provider = newRequestProvider(`${BASE_URL}`);
Windsurf: Refactor | Explain | Generate JSDoc |
export const listCategories = async (userId: string, categoryName?: string) => {
return await provider.get<CategorieType[]>(
`categories?user_id=${userId}${categoryName ? `&name=${categoryName}` : ""}`
);
};
Windsurf: Refactor | Explain | Generate JSDoc |
export const createCategory = async (payload: CreateCategoryType) => {
return await provider.post(
`categories`,
{
body: {
...payload,
},
}
);
};
master*
Launchpad
Jira: Heitor Moreira:
No active issue
Bitbucket: Heitor Moreira:
0 1
You, há 10 horas
Ln 1, Col 1
UTF-8
LF
TypeScript
Go Live
Windsurf: Login
Prettier

FLUXO DA APLICAÇÃO

```java
package com.ti2cc.api.factory;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class PostgresFactory {
    public static Connection getConnection() {
        try {

            String url = "jdbc:postgresql://db.ewyqqxoqofphrhjwehim.supabase.co:5432/postgres";
            String user = "postgres";
            String password = "n64n5o4n5lj4n6ij4";

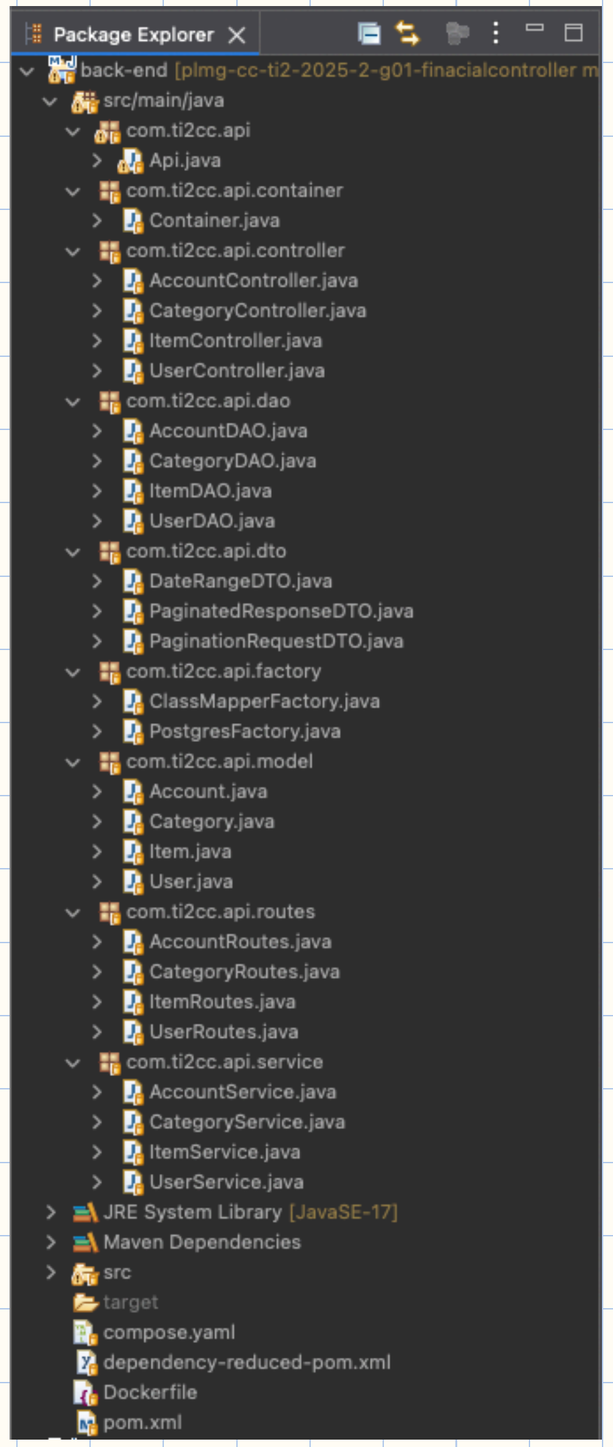
            return DriverManager.getConnection(url, user, password);
        } catch (SQLException e) {
            throw new RuntimeException("Fail to establish a connection with postgres DB: ", e);
        }
    }
}

```

ESTRUTURA E ORGANIZAÇÃO DO PROJETO

src/main/java: Contém todo o código-fonte do back-end da aplicação.

Container: Inicializa e gerencia os componentes principais do sistema
Controller: Controla as requisições da API e repassa para os serviços
DAO: Executa as operações de acesso e persistência no banco de dados
Factory: Cria e gerencia conexões e mapeamentos de classes
DTO: Define objetos de transferência de dados
Model: Representa as entidades principais (Account, Category, User)
Routes: Define as rotas da API (endpoints)
Services: Contém as regras de negócio e validações do sistema

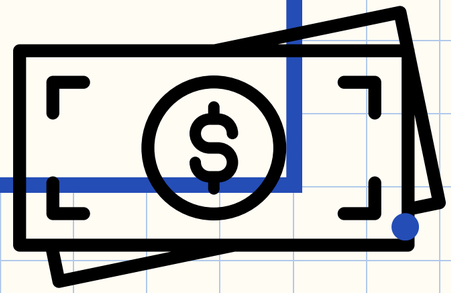


RECURSOS DE IA

- O sistema utiliza modelos de embeddings para interpretar o significado de frases como:

“Gastei 45 reais no iFood ontem” → Categoria: Alimentação

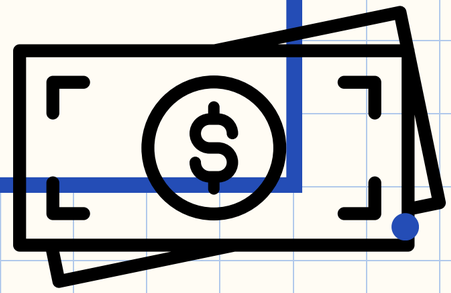
- O recurso de Speech-to-Text transforma comandos de voz em texto, permitindo registrar gastos falando.



PROPOSTA DE SISTEMA INTELIGENTE

- Para o usuário:
  - Controle financeiro simplificado por voz e chat.
  - Categorização automática e precisa de despesas.
  - Relatórios claros e personalizados com insights inteligentes.

- Para o Mercado/Investidores:
  - Solução diferenciada com foco em usabilidade e automação financeira.
  - Potencial integração com bancos, fintechs e assistentes virtuais.



ARQUITETURA DE SOLUÇÃO DE IA

- **Serviços Específicos:**
  - **OpenAI Embeddings API** – para categorização semântica de transações financeiras.
- **Tipo de Aprendizado:**
  - **Pré-treinado e online** – uso de modelos já treinados em larga escala, integrados via API.

- **Entradas e Saídas do Modelo:**
  - Entrada: Texto digitado pelo usuário no chat ou fala convertida em texto (Speech-to-Text).
  - Saída: Objeto JSON com dados como:
    - Categoria identificada
    - Valor e descrição processados
    - Resposta textual para o chat



OBRIGADO